

Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

fastEmbedR: Multicore and GPU UMAP and Compact-Support t-SNE in R

Moussa Kassim Mohamed[1,2,*] **Martin Ocharo**[1,2,*]
Giulia Canarutto[3,4] Dalia Ahmed[1] Dupe Ojo[1]
Alessia Vignoli[5,6] Leonardo Tenori[5,6,†]
Dinesh Gupta[7] Silvano Piazza[3,8]
Stefano Cacciatore[1,2,†]

Abstract

Uniform manifold approximation and projection (UMAP) and t-distributed stochastic neighbor embedding (t-SNE) are widely used for exploratory visualization, but large data sets make neighbor search, initialization, graph construction, and layout optimization expensive. **fastEmbedR** is an MIT-licensed R package that implements fuzzy UMAP and a compact-support interpolation-based t-SNE variant, with candidate width equal to perplexity, on multicore CPU, Apple Metal, and NVIDIA CUDA. Matrix, float32, and reusable nearest-neighbor inputs share strict backend reporting. The package also provides randomized principal component analysis (PCA), reusable weighted graphs, community detection, optional landmark fitting, and fixed-reference projection. Experimental file-backed interfaces support streamed PCA and explicitly prepared full-row graph optimization without requiring an R matrix to hold the input. These routes are evaluated separately from established in-memory workflows; full-row output is not by itself evidence of a memory-boundary or multi-GPU speedup. Component tests compare graph weights, affinities, forces, gradients, and trajectories with independent references. The benchmark design evaluates complete public calls separately from direct-Python fits and records runtime, memory, observed neighbor recall, and embedding quality. The available full-data, 11-data-set workflow campaign is diagnostic because its final audit failed. It does not establish release-level speedups. Source, documentation, and reproducible benchmark code are publicly available.

Keywords: dimensionality reduction, GPU computing, Metal, CUDA, PCA, t-SNE, UMAP.

1. Introduction

Uniform manifold approximation and projection (UMAP) and t-distributed stochastic neighbor embedding (t-SNE) are standard tools for inspecting high-dimensional data in genomics, cytometry, image analysis, and machine learning (van der Maaten and Hinton 2008; McInnes, Healy, and Melville 2018; Kobak and Berens 2019). Their output is a low-dimensional layout that aims to retain local relationships from the original feature space. The computational burden is not limited to the final optimization. A complete workflow also includes preprocessing, k-nearest-neighbor (KNN) search, graph or affinity construction, initialization, and data movement between the R process and an accelerator.

Several implementations address individual parts of this problem. Barnes-Hut t-SNE reduces repulsive-force complexity (van der Maaten 2014); interpolation-based t-SNE uses a regular grid and fast Fourier transform (FFT) convolutions (Linderman, Rachh, Hoskins, Steinerberger, and Kluger 2019); and GPU implementations accelerate both t-SNE and UMAP (Chan, Rao, Huang, and Canny 2018; Nolet, Lafargue, Raff, Nanditale, Oates, Zedlewski, and Patterson 2020). In R, **Rtsne** and **uwot** provide mature CPU workflows, while FIt-SNE, Python openTSNE, umap-learn, and RAPIDS cuML provide additional CPU or CUDA routes. A user nevertheless often has to combine several packages or language environments, materialize the same neighbor graph repeatedly, and accept different data types and timing boundaries.

fastEmbedR addresses this systems problem with one native R interface for multicore central processing units (CPUs), Apple Metal, and NVIDIA Compute Unified Device Architecture (CUDA). Its principal contributions are software-engineering rather than new UMAP or t-SNE mathematics:

- package-owned float32 fuzzy UMAP and compact-support t-SNE implementations, including interpolation-based repulsion, with strict backend reporting;
- matrix and precomputed-neighbor interfaces that reuse native nearest-neighbor results and avoid unnecessary host-device transfers;
- landmark models and fixed-reference transformation for new samples;
- shared CPU, Metal, and CUDA validation tests for affinities, graph weights, forces, optimization, precision, and difficult inputs; and
- typed results carrying resolved parameters, precision, backend, and timing information.

Supporting utilities provide backend-native randomized PCA for preprocessing and t-SNE initialization and native weighted graph construction with Louvain, Leiden, and Walktrap community detection. These utilities complete common embedding workflows but are not the basis of the principal embedding claims.

The package is intentionally not an API clone of any one reference implementation. It follows the published UMAP objective and the interpolation-based t-SNE optimizer, while its production t-SNE affinity uses compact support with one non-self candidate neighbor per unit of perplexity. This policy differs from conventional sparse t-SNE workflows with wider candidate sets. Fuzzy UMAP is the default. A binary graph mode is retained as a separately labeled adjacency sensitivity analysis that can emphasize cluster boundaries, but it changes the graph objective and is not a replacement for standard fuzzy UMAP.

Table 1: Core embedding capabilities of related software. Experimental file-backed routes are excluded.

Software	Method	Multi-CPU	CUDA	Metal	Landmarking	KNN input	3D output
Rtsne 0.17	t-SNE	Yes	No	No	No	Yes	Yes / –
FIt-SNE 47ff14f	t-SNE	Yes	No	No	No	Limited	Yes / –
openTSNE 1.0.4	t-SNE	Yes	No	No	Manual	Yes	Yes / –
uwot 0.2.5	UMAP	Yes	No	No	Manual	Yes	– / Yes
umap 0.2.10.0	UMAP	No	No	No	Manual	Yes	– / Yes
umap-learn 0.5.12	UMAP	Yes	No	No	Manual	Yes	– / Yes
RAPIDS cuML 26.06	UMAP, t-SNE	No	Yes	No	UMAP only	Yes	No / Yes
fastEmbedR	UMAP, t-SNE	Yes	Yes	Yes	Built-in	Yes	Yes / Yes

The 3D column is ordered t-SNE / UMAP; a dash means the method is not implemented. FIt-SNE and openTSNE use a Barnes–Hut route for 3D output rather than FFT interpolation. cuML's 3D output and reference projection are UMAP-only. fastEmbedR supports three-dimensional t-SNE and UMAP on CPU, CUDA, and Metal for ordinary matrix or precomputed-neighbor fits. Three-dimensional GPU landmark refinement and three-dimensional full-graph CUDA t-SNE are not available.

Table 1 summarizes capabilities of the listed versions. Multicore CPU indicates user-selectable parallel CPU work; KNN input means that an embedding can consume precomputed nearest neighbors. For landmarking, "manual" means fitting a selected reference subset and projecting the remainder through a separate public transform method, whereas "built-in" includes landmark selection and projection.

This article combines the software description, implementation details, usage examples, numerical validation, workflow benchmarks, and limitations. UMAP and t-SNE remain the central embedding contribution; PCA and graph clustering are evaluated as integrated computational stages that let an analysis remain within the same typed, backend-aware workflow.

2. Software architecture

2.1. Execution model and ownership boundary

The principal execution path begins with a matrix that may be centered, scaled, or reduced by randomized PCA. Nearest-neighbor search provides the sparse local relationships required by both methods. UMAP turns these relationships into a weighted graph; t-SNE turns them into sparse probabilities. A backend-specific optimizer then produces the final layout. Precomputed neighbors can enter after the search stage, and landmark models separate reference fitting from projection of omitted or genuinely new rows.

The required R layer imports **faissR**, **Rcpp**, **withr**, and the **methods**, **parallel**, and **utils** packages distributed with R. The native build uses **Rcpp** and **faissR** headers. CPU exact search is package-native; CPU HNSW calls the versioned **faissR** C API, which uses FAISS internally, without calling a **faissR** R function. Other CPU numerical routines use R's configured BLAS, LAPACK, and Fortran runtime. Matrix workflows construct neighbors, whereas the precomputed-neighbor interfaces define an explicit boundary at which a caller can supply

indices and distances. The selected metric, neighborhood size, search engine, observed recall, and backend are retained for reproducibility. Search time remains part of complete public-workflow timing. For CPU HNSW, the reported campaign used a predeclared policy based on matrix shape and neighborhood size. Its nominal 0.99 recall target is a policy objective, not a guarantee for a new data set; benchmark observations did not select the settings. Recall is checked on fixed exact-reference queries after fitting. A comparative study of nearest-neighbor algorithms is outside the scope of this article, but the accuracy of the search used by each reported workflow is measured because it can affect both runtime and embedding quality. Metal and CUDA builds link optional accelerator libraries; they do not redistribute them. Copyright and third-party notices are retained in the package source.

2.2. Accelerator library boundaries

CUDA uses cuVS (built on RAFT) for matrix-derived neighbors, cuFFT for t-SNE repulsion, and cuBLAS/cuSOLVER/cuRAND for PCA (NVIDIA 2026b). UMAP and t-SNE optimization remains package-owned. Experimental coordinated multi-GPU layouts use NCCL (NVIDIA 2026a); partitioned KNN and PCA are distinct operations and do not prove multi-GPU optimizer acceleration. The supplement maps each library to its stage.

Explicit backend requests are strict. If `backend = "metal"` or `backend = "cuda"` is unavailable, the function raises an error instead of silently reporting CPU work as accelerated work. When `backend = NULL`, the session option and environment-variable policy are consulted, with CPU as the final default. Successful results record both the requested and observed backend; unavailable accelerator requests fail before a result is returned.

2.3. Data representation and memory

The production numerical payload uses float32. Ordinary R matrices require one controlled conversion at the native boundary. Objects from **float** are read directly. Neighbor distances, affinities, graph weights, layouts, gradients, gains, and update buffers remain float32; indices and compressed sparse row (CSR) offsets use int32. A float32 input returns a float32 layout, whereas an ordinary matrix returns a double matrix only at the final R boundary.

The memory reduction is smaller than 50 percent in a complete process because object headers, labels, int32 graph structure, search indexes, FFT padding, spectra, and allocator workspaces are not double-precision payloads. CUDA keeps eligible neighbor, graph, affinity, layout, and optimizer buffers on-device. Metal uses unified memory, so separate host and device peaks cannot be added meaningfully. The result records persistent and temporary native-buffer estimates when available.

2.4. Experimental file-backed workflows

The default matrix interface is distinct from experimental file-backed input. `massive_matrix()` describes a row-major float32 file; `pca()` streams it in bounded blocks. `umap()` and `tsne()` either fit landmarks and project omitted rows or optimize all rows on a prepared file-backed graph. Landmark projection is an approximation and is never selected silently. Full-graph optimization needs an explicitly prepared graph and initialization; it need not use exact neighbors. Current Deep1B restart support covers KNN construction, not interrupted UMAP, t-SNE, or clustering optimization. Memory budgets limit algorithm buffers, not whole-process

Table 2: Canonical public API. CPU, Metal, and CUDA are available where the package was compiled with the corresponding backend.

Function	Purpose
`tsne()`, `umap()`	Complete matrix-to-layout workflows.
`tsne_knn()`, `umap_knn()`	Layout from reusable neighbor indices and distances.
`precompute_knn()`	Backend-native non-self neighbor search.
`pca()`	Randomized PCA and optional t-SNE-scaled initialization.
`select_landmarks()`	Reusable landmark/reference selection.
`project_landmark_model()`	Project omitted or new observations into a fixed reference.
`transform_tsne()`	Fixed-reference t-SNE transformation.
`knn_graph()`, `graph_cluster()`	Secondary graph construction and Louvain, Leiden, or Walktrap clustering.

RSS or disk cache. Metal full-graph t-SNE and clustering remain unsupported. The file-backed modes and their evidence gates are detailed in the supplement.

2.5. Public interface

Table 2 maps the stable interface. Generic names are always shown with namespace qualification in mixed-package sessions, for example `fastEmbedR::umap()` and `fastEmbedR::pca()`.

The principal embedding functions return an S3 `fastEmbedR_embedding` object with coordinates, resolved parameters, requested and observed backend, precision, stage timings, and optional neighbor state. Methods for `print()` and `plot()` provide concise summaries. Neighbor, PCA, landmark, graph, and clustering results have dedicated classes. The current source accepts three-dimensional t-SNE and fuzzy UMAP on CPU, CUDA, and Metal for ordinary fits. The evaluated image predates native 3D GPU support, so its benchmarks do not validate those newer routes.

3. Algorithms and backend implementations

3.1. UMAP

UMAP constructs a fuzzy graph from local distances and optimizes a low-dimensional cross-entropy objective (McInnes *et al.* 2018). For observation i, the package estimates local connectivity ρ_i and scale σ_i, forms directed membership strengths, and combines reciprocal memberships as

$$w_{ij} + w_{ji} - w_{ij}w_{ji}.$$

The resulting CSR graph stores the row pointer, column index, weight, and edge sampling schedule once. Fuzzy weighting is the standard default. Binary mode sets unit weight on the symmetric neighbor union; it can make adjacency-defined boundaries visually sharper, but it changes the objective and its quality varies by data set.

CPU, Metal, and CUDA implementations retain graph and optimizer buffers across epochs; CUDA fuses row updates and captures repeated work in a CUDA Graph. Public controls include neighborhood size, metric, graph mode, preprocessing, backend, seed, output dimension, and CPU cores. Remaining optimizer values follow the recorded release policy.

3.2. Compact-support interpolation-based t-SNE

t-SNE converts local high-dimensional distances into probabilities p_{ij} and minimizes Kullback-Leibler (KL) divergence from these probabilities to the low-dimensional Student distribution (van der Maaten and Hinton 2008). **fastEmbedR** implements sparse Gaussian affinity construction, early exaggeration, momentum, adaptive gains, gradient clipping, and interpolation-based FFT repulsion for two-dimensional layouts in the evaluated image (Linderman *et al.* 2019).

The package uses compact affinity support: perplexity P selects $\lceil P \rceil$ non-self candidate neighbors. This policy is shared by the matrix, precomputed-neighbor, landmark, and transformation interfaces and by the CPU, Metal, and CUDA backends. The candidate width, achieved perplexity, optimizer schedule, learning rate, exaggeration, and repulsion method are returned with the fit. For a conditional probability vector over k candidates, $H(p_i) \leq \log(k)$. Consequently, when $k = P$, the target entropy $\log(P)$ lies at the boundary and is attained exactly only by uniform conditional weights. The finite solver tolerance produces nearly uniform, rather than mathematically identical, weights. High-dimensional distances therefore determine candidate membership much more strongly than relative mass within a candidate row; reciprocity during symmetrization can still differentiate the final undirected edge weights. The production API does not expose a wider affinity-support mode; therefore, all cross-backend t-SNE validation in this article evaluates this same compact policy, including the CUDA backend.

The two-dimensional FFT grid follows a recorded size- and backend-dependent policy. cuML and FIt-SNE expose different grid controls, so their FFT configurations are not assumed identical. CPU, Metal, and CUDA reuse plans and optimizer state; CUDA captures repeated iteration work. Public calls synchronize before timing stops. The evaluated image does not validate native three-dimensional accelerator fits.

3.3. PCA and initialization

Randomized singular value decomposition approximates a low-rank subspace using random projection and power iterations (Halko, Martinsson, and Tropp 2011). The public `fastEmbedR::pca()` function returns scores, loadings, singular values, centering and scaling vectors, decomposition metadata, and the observed backend. With `tsne_init = TRUE`, it also centers the requested low-dimensional scores and rescales them so that the largest component standard deviation is 10^{-4}, which is directly accepted by `fastEmbedR::tsne()` and `fastEmbedR::tsne_knn()`.

CPU PCA uses a blocked float32 randomized range finder followed by a small dense decomposition. Metal uses a float32 block-subspace randomized decomposition with Metal Performance Shaders. CUDA selects between native float32 randomized SVD and resident RAPIDS RAFT truncated singular value decomposition from matrix shape, requested rank, and estimated arithmetic cost. The typed result includes scores, loadings, singular values, preprocessing vectors, optional test-set projections, and the observed backend. Comparisons with **irlba** (Baglama, Reichel, and Lewis 2026) assess performance and agreement between two approximate methods rather than treating either as an exact reference. The PCA alignment study evaluates CPU, the tested M3 Metal system, and CUDA. Agreement is measured after Procrustes or subspace alignment because truncated decompositions can return equivalent bases with different signs or rotations. When PCA is requested inside an embedding workflow, its preprocessing and transfer costs belong to the complete-call time; a standalone PCA

benchmark is reported separately. The shape-dependent CUDA choice is recorded for each fit, because a faster truncated decomposition on one matrix shape need not remain faster on another.

3.4. Landmarks and new observations

Landmarking is explicit. A reference subset is selected by a seeded projection-diversity policy or supplied by the user. The ordinary `fastEmbedR::tsne()` or `fastEmbedR::umap()` implementation embeds that subset. Remaining rows query the fixed reference, receive weighted or affine-interpolated coordinates, and can undergo a short row-restricted refinement while reference coordinates are restored after each update.

The package distinguishes reconstruction of omitted rows from prediction of genuinely unseen observations. In held-out transformation, query rows do not participate in landmark selection, reference preprocessing, neighbor search among reference rows, initialization, or stopping. t-SNE query probabilities are optimized with the reference fixed (Poličar, Stražar, and Zupan 2021); UMAP projects queries through reference neighbors and optional fixed-reference refinement. The returned model records landmark indices, preprocessing, neighborhood policy, graph or affinity policy, backend, and stage timings. Landmarking is a scalability approximation, not a guarantee of faster execution or unchanged quality.

3.5. Graph construction and community detection

The same sparse-neighbor infrastructure constructs undirected weighted graphs from a feature matrix or reusable KNN object. Edge weights can encode shared-neighbor Jaccard similarity, inverse distance, or binary adjacency; mutual-neighbor filtering and weight pruning alter the resulting graph and must be reported with it. Reusing a graph permits community analyses without another neighbor search, although graph construction remains a distinct timed stage.

Package-native Louvain and Leiden support CPU, Metal, and CUDA local movement, while exact Walktrap is CPU-only (Blondel, Guillaume, Lambiotte, and Lefebvre 2008; Traag, Waltman, and van Eck 2019; Pons and Latapy 2006). A clustering fit reports membership, modularity, connectivity diagnostics, backend, and run parameters. Agreement with **igraph** is assessed on the identical stored weighted graph, with adjusted Rand index (ARI) for membership; stochastic algorithms need not return identical partitions. These utilities support interpretation after embedding but are not evidence for UMAP or t-SNE correctness. Optional graph-supported layout merging combines communities only when original-graph connectivity and local UMAP or t-SNE edge continuity agree. It is a PAGA-inspired exploratory rule (Wolf, Hamey, Plass, Solana, Dahlin, G"ottgens, Rajewsky, Simon, and Theis 2019), not PAGA or evidence from plot proximity alone; the controlled example is in the supplement.

4. Using fastEmbedR

4.1. Installation and backend inspection

A CPU build needs R, **faissR**, **Rcpp**, **withr**, and a C++17 toolchain. Metal requires the Apple SDK; CUDA requires a compatible driver and toolkit, with cuVS for matrix-derived neighbors and NCCL for coordinated multi-GPU layouts. Explicitly requested missing components

cause build or runtime errors. Release-tested commands and optional library roles are in the installation guide and supplement.

Session defaults use `options(backend = "cuda")` and `options(n.cores = 4L)`; explicit arguments take precedence. CPU is the final default, and successful results record the observed backend.

The benchmark preflight executes this CPU example:

```
set.seed(4)
x <- matrix(rnorm(128 * 8), 128, 8)
options(backend = "cpu", n.cores = 2L)
fit <- fastEmbedR::umap(x, n_neighbors = 15, seed = 4)
knn <- fastEmbedR::precompute_knn(x, k = 15)
from_knn <- fastEmbedR::umap_knn(
    knn$indices, knn$distances, seed = 4)
```

The matrix fit returns `fit$layout`; `from_knn` is a two-column layout computed from the reusable neighbor result. An explicit `backend` argument can override the session default.

4.2. Matrix and precomputed-neighbor workflows

The matrix interfaces `tsne()` and `umap()` perform preprocessing, neighbor search, initialization, and optimization. The corresponding `tsne_knn()` and `umap_knn()` interfaces consume reusable indices and distances, avoiding repeated search in seed, optimizer, or graph-mode comparisons. The CUDA route can retain the neighbor object on the device and consume it directly during affinity or graph construction.

4.3. PCA and landmark models

The `pca()` interface returns ordinary scores and, with `tsne_init = TRUE`, a scaled t-SNE initialization. Landmarking is enabled within `tsne()` or `umap()` by supplying a fraction, count, or row indices to `landmarks`; the default `FALSE` fits all observations. The fitted result contains a reusable model accepted by `project_landmark_model()` for independently held-out rows. Reference coordinates remain immutable during projection.

With file-backed input, `pca()` streams scores when `massive` is `out_of_core`. Setting `massive` to `landmark` in `umap()` or `tsne()` fits a chosen reference subset and writes projected coordinates to disk. The prepared-graph modes instead update all observations. The experimental vignette documents file layout, memory controls, checkpointing, and unsupported backend combinations.

5. Validation

5.1. Validation strategy

Final t-SNE coordinates are non-convex and need not be identical across parallel implementations. Correctness was therefore tested below the final layout as well as at the layout level. Tests used exact high-dimensional neighbors and common initial coordinates where the compared APIs permitted them. UMAP validation compared edge sets, fuzzy weights,

Table 3: Component-level t-SNE validation. Relative L2 error is measured against the stated independent reference.

Component	Acceptance threshold	Maximum observed
Float32 affinity versus float64	2×10^{-6} absolute	2.06×10^{-8}
Exact attractive force	5×10^{-5} relative	4.12×10^{-7}
Exact repulsive force	5×10^{-5} relative	1.68×10^{-7}
Finite-difference gradient	2×10^{-4} relative	1.27×10^{-7}
FFT repulsion, grid 128	10^{-3} relative	3.37×10^{-5}
CPU-CUDA FFT first step	2×10^{-3} relative	3.80×10^{-8}
Common-affinity KL	2×10^{-5} absolute	1.21×10^{-6}
Pathological inputs	finite result or explicit error	15 of 15 passed

aligned coordinates, and embedding neighborhoods. t-SNE validation compared conditional and symmetrized affinities, exact attractive and repulsive forces, finite-difference gradients, FFT-grid force convergence, one-step updates, objective trajectories, final KL divergence, aligned coordinates, and embedding neighborhoods.

PCA tests compare aligned scores, principal angles, captured variance, and test-set projection. Graph tests use canonical clique graphs, stochastic block models, isolated vertices, disconnected candidate communities, high-degree vertices, and fixed real-data graphs. Louvain and Leiden are compared with **igraph** modularity and membership; Walktrap is compared with the **igraph** Pons–Latapy implementation.

Pathological embedding tests included duplicated observations, zero distances, disconnected graphs, coincident coordinates, extreme finite coordinates, and nonfinite input. CPU float32 forces were also compared with an independent float64 oracle. Fixed seeds make the deterministic one-thread CPU route repeatable, but do not guarantee bitwise Metal or CUDA identity because atomic update order can differ.

5.2. Affinity, graph, and force agreement

On a fixed 2,000-point MNIST subset, fuzzy UMAP edge Jaccard agreement with a **uwot** similarity graph was 1.000000; weight Pearson and Spearman correlations were 0.999990 and 0.999977. On common compact t-SNE support, edge Jaccard and Pearson weight agreement with Python openTSNE were 1.000000. The Spearman value of 0.852 arose from rank changes among numerical near-ties: all matched weights differed by less than 10^{-7} and the maximum absolute difference was 1.66×10^{-8}. This validates arithmetic on the supplied compact support, rather than equivalence to a conventional wider-support t-SNE affinity model.

The fixed-input affinity characterization confirms the mathematical consequence of this policy. At each tested perplexity, the achieved perplexity equaled the supplied support width, normalized entropy was 1, and the conditional coefficient of variation was numerically zero. Distances therefore select the neighbors, while reciprocity in symmetrization provides most of the remaining edge-weight variation. This is a deliberate compact approximation and is not presented as affinity-equivalent to conventional wider-support t-SNE.

Table 3 reports lower-level t-SNE tests. FFT force error fell from 5.07×10^{-4} at grid 32 to 7.68×10^{-6} at grid 256. The CUDA validation exercises the production FFT path; it does not report an unsupported exact-repulsion CUDA route as a passing test.

Table 4: Fixed-input compact-support t-SNE final-layout validation. Seed-42 diagnostic values at 1,000 normal iterations and a fixed 256-cell FFT grid. Python openTSNE supplies the common reference fit. USPS trustworthiness was present in the saved results.

Data set	Implementation	Common KL	Trustworthiness	Preserve@30
Fashion-MNIST	Python openTSNE	1.2842	0.9819	0.5765
	fastEmbedR CPU	1.2835	0.9818	0.5784
	fastEmbedR CUDA	1.2829	0.9819	0.5783
MNIST	Python openTSNE	1.7367	0.9557	0.5228
	fastEmbedR CPU	1.7403	0.9539	0.5211
	fastEmbedR CUDA	1.7373	0.9557	0.5210
MetRef	Python openTSNE	1.8354	0.8460	0.4352
	fastEmbedR CPU	1.8476	0.8558	0.4427
	fastEmbedR CUDA	1.8378	0.8528	0.4405
USPS	Python openTSNE	1.6906	0.9577	0.5203
	fastEmbedR CPU	1.6889	0.9578	0.5229
	fastEmbedR CUDA	1.6900	0.9580	0.5218

5.3. Final-layout agreement

Compact-support validation used common initialization and affinities for 2,000 observations from Fashion-MNIST, MNIST, MetRef, and USPS. The archived 1,000-normal-iteration outputs used seed 42 and a 256-cell FFT grid in Table 4. Other seed runs stopped at 750 normal iterations; they cannot be counted as 1,000-iteration replications. The corrected benchmark schedules seeds 4, 17, and 42 at 1,000 normal iterations on the same grid. Until that run is complete, the table is a diagnostic single-seed comparison, not evidence of between-seed stability. Metal was not part of this Linux experiment and is not inferred from it.

Across three FFT grid sizes at seed 42 and 1,000 iterations, median Procrustes correlations with Python openTSNE were 0.998, 0.930, 0.778, and 0.995 for CPU on Fashion-MNIST, MNIST, MetRef, and USPS; the corresponding CUDA values were 0.998, 0.939, 0.764, and 0.992. Neighborhood agreement at 30 ranged from 0.672 to 0.936 for CPU and from 0.650 to 0.935 for CUDA. The lower coordinate agreement on MetRef coexisted with closely matched KL and local-neighborhood quality, illustrating why non-convex t-SNE validation cannot rely on coordinate correlation alone.

With common precomputed neighbors and matched seeds, fixed-input UMAP CPU-CUDA Procrustes correlation had median 0.962 across 33 data-set–seed pairs (range 0.776–0.996). Neighborhood agreement at 30 had median 0.812 (range 0.473–0.900). The lowest agreements occurred for ImageNet and USPS; their data-set medians were 0.904/0.474 and 0.799/0.574 for Procrustes/neighborhood agreement. These results establish behavioral agreement rather than bitwise identity and retain the weaker cases instead of reporting only aggregate medians.

5.4. Precision boundary

Using the same exact neighbors, initialization, and seed, ordinary double matrices and direct **float** inputs produced identical affinity or graph weights, aligned layouts, neighborhood metrics, and t-SNE KL in the focused boundary test. Runtime differences were small and not consistent across objectives. The result supports numerical agreement and copy avoidance, not a universal speedup for the float32 input class.

Table 5: Benchmark data sets and fitted row counts in workflow campaign `20260930T203319Z`. All rows were fitted. ImageNet features are used primarily as a scalability and memory stress test rather than a two-dimensional class-separation task.

Data set	Source rows	Fitted rows	Variables	Domain
COIL20	1,440	1,440	16,384	Object images
USPS	11,000	11,000	256	Handwritten digits
MNIST	70,000	70,000	784	Handwritten digits
Fashion-MNIST	70,000	70,000	784	Clothing images
MetRef	873	873	375	Metabolomics
mass41	965,282	965,282	14	Mass cytometry
flow18	1,000,021	1,000,021	11	Flow cytometry
ImageNet features	1,281,167	1,281,167	1,024	DINOv2 features
FlowRepository	5,220,347	5,220,347	32	Mass cytometry
Tabula Muris	100,102	100,102	50	Single-cell RNA-seq PCs
Macosko retina	44,808	44,808	50	Single-cell RNA-seq PCs

6. Benchmark design

6.1. Data sets and hardware

The diagnostic workflow campaign covered 11 external data sets (Table 5): image data, metabolomics, cytometry, single-cell RNA sequencing, and foundation-model image features. No benchmark matrix or label vector is distributed with the package. The package vignette uses only `datasets::iris`.

Linux CPU and CUDA jobs ran on the University of Cape Town high-performance computing facility. CPU nodes used Intel Xeon Gold 6442Y processors; the main comparison allocated four cores. CUDA jobs used one NVIDIA L40S with 46,068 MiB memory and compute capability 8.9. The environment contained Debian 13, R 4.6.1, OpenBLAS, **uwot** 0.2.5, **Rtsne** 0.17, **umap** 0.2.10.0, **reticulate** 1.46.0, and **float** 0.3.3. Metal tests used an Apple M3 with 8 GB unified memory. Accelerator timing was synchronized before stopping the timer.

The result manifest identifies every comparator independently of its display label. Python comparisons used Python 3.12.13, NumPy 2.2.6, openTSNE 1.0.4, scikit-learn 1.9.0, umap-learn 0.5.12, and RAPIDS cuML 26.6.0. FIt-SNE 1.2.1 was built from commit `47ff14f1de fc1ff3a8065c8b7baaf45c33e7e0b2`; its executable has SHA-256 `b14ce0ba5310adf0a120 7d2c114f94038812cd20fb93ccc76efa599528ddaf91`. Missing comparator identity is treated as an unavailable benchmark condition and is not eligible for comparative ratios. The publication-data builder also requires a passing campaign audit. That audit checks expected method–data-set combinations, successful statuses, nonempty layout and timing artifacts, finite elapsed times, and agreement between each method's declared family and its recorded result. NOMAD, when enabled in the frozen CUDA comparator run, is a separate projection objective. It is shown descriptively, not counted as a UMAP or t-SNE method or included in their speedup denominators.

6.2. Timing boundaries and metrics

The primary benchmark measures the complete public function: input preparation, transfer, neighbor search and any tuning actually performed, initialization, graph or affinity construction, optimization, and returned-layout materialization. These are workflow-level comparisons under

each package's reported configuration, not isolated optimizer comparisons. R-mediated Python total calls, direct-Python fit time, and direct-process wall time remain distinct timing scopes.

Each benchmark runs in an isolated worker. Host peak resident set size (RSS) is obtained from GNU `time` when available; on heterogeneous Slurm nodes without that utility, the wrapper records elapsed wall time itself and reads the completed batch step's MaxRSS from `sstat`. The measurement source is stored with every row, and absence of both measurements is explicit rather than interpreted as zero. CUDA calls synchronize before timing stops; direct-Python calls materialize the returned layout on the host for the same reason. GPU memory is sampled every 0.2 seconds with `nvidia-smi` on an exclusive device, recording the device-wide baseline before launch, the peak, and the nonnegative peak-minus-baseline value. The baseline includes any initialized runtime or allocator cache. Absolute and baseline-adjusted GPU values are retained because container PID namespaces do not provide reliable per-process device accounting.

Controlled comparisons separately reuse neighbors, support, initial coordinates, metric, seed, and iteration or epoch count wherever the APIs allow. A method that cannot satisfy a control is marked unmatched and is not used for a matched-boundary speed ratio. In each **fastEmbedR** quality-diagnostic workflow, the neighbor object is retained after timing has stopped. Fixed stratified query rows are compared with exact neighbors computed against the complete benchmark matrix. These queries evaluate recall; they do not select the fixed-policy HNSW settings. A data-aware policy requires disjoint tuning and holdout queries: settings must be frozen before holdout evaluation, while calibration, exact-reference work, and final fitting all count toward end-to-end time. Among settings meeting the prespecified recall bound, selection minimizes measured build-plus-query time for a declared query count; small timing differences require repeated same-host confirmation. Selecting the smallest HNSW parameters is not equivalent to selecting the fastest workflow. Search-only and amortized timings are separate scopes. Fixed and calibrated policies must be reported as different arms, with a recall miss visible rather than silently replaced by a faster result. Query-level holdout tests within-data-set calibration; it does not validate a policy designed using that same data set. Cross-data-set claims require freezing the policy before evaluation on previously unused data sets. Mean, median, fifth-percentile, and minimum recall at the fitted neighborhood size are reported with the search engine, exact or approximate status, target recall, and resolved HNSW or IVF parameters. Exact-reference computation for fixed-policy diagnostics and device-to-host materialization are excluded from the recorded workflow runtime. The separate cuVS search diagnostic is not divided by a fit time. Stage times in a workflow comparison must come from that same timed call.

Quality metrics include trustworthiness (Venna and Kaski 2001), overlap between high- and low-dimensional neighbor sets, silhouette when labels are available (Rousseeuw 1987), embedding-space label KNN accuracy, t-SNE KL, affinity agreement, fuzzy-graph agreement, and Procrustes-aligned layout similarity. Labels are never used for fitting. Comparator metrics use deterministic, stratified samples of 500–2,000 observations per data set; backend-quality tests may use larger fixed samples. The one-based row identifiers are stored in both RDS and CSV form, and the CSV SHA-256 is copied into every result row. The same observations are therefore reused across methods, backends, timing repetitions, and embedding seeds.

Embedding seeds 4, 17, and 42 quantify seed-associated variation and are not treated as independent timing repetitions. Deterministic initialization and optimization can produce identical layouts across these seeds; such rows do not constitute independent stability observations. The

complete-workflow protocol specifies one excluded warm-up and five same-seed repetitions for R and direct-Python calls. The September 30 workflow campaign contains these repetitions for timing-eligible methods. Long jobs completed once under the two-hour method budget are identified separately; the final audit failed. The matched t-SNE embedding-only study used five CPU and Python repetitions and ten CUDA repetitions after warm-up. Timing variation and embedding-seed variation are reported separately. For paired workflow summaries the data set is the independent comparison unit; timing repetitions, query rows, and rasterized pixels are not additional data sets. A single Deep1B window supports a case-specific measurement, not a cross-data-set confidence interval.

6.3. File-backed evaluation design

The experimental study uses a fixed Deep1B row window beginning at source row 1,000,001, with 96 float32 variables and fitted prefixes of 1 and 10 million rows. The 1-million-row arm pairs ordinary matrix input and file-backed calls on identical observations. File-backed PCA, full-row nearest-neighbor search, t-SNE, and UMAP are requested at each size. CPU and multi-GPU arms also request graph clustering. CPU allocations request four, eight, or 16 cores. CUDA allocations request one or two L40S devices for file-backed work; the ordinary-matrix comparison requests one device. These are requested arms. The one- and 10-million-row two-device full-row arms completed execution, but did not pass the strict sampled per-query neighbor-recall gate and are not qualified comparisons. The supplement lists the requested arms and their capability boundaries. A 100-million-row two-GPU fit and a final billion-row two-GPU fit are deferred until end-to-end restart and quota audits pass; they are not submitted as part of the current matrix.

The full-row arm uses the same existing CPU HNSW or CUDA cuVS search route as the selected backend. It is not a landmark reconstruction: every fitted row receives optimized coordinates. Approximate-neighbor recall is measured on fixed exact-reference queries, with effective search settings and build, query, and audit time reported separately. A sampled mean above 0.99 does not establish a per-query 0.99 guarantee or independent tuning holdout. The campaign also records successful output-row count, stage times, peak host RSS, sampled per-device GPU memory, and output-file size. The classic and file-backed arms are compared only at one million rows and only when both finish with the same output scope and quality assessment. Deep1B has no supplied class labels for this study; full-row density plots therefore use no inferred class or community colors.

The largest current-matrix input window is 3.84 GB. Its size alone does not establish that the input exceeded device or host memory; graph and working-set requirements must be measured separately. An allocation of multiple GPUs does not demonstrate distributed work: the device-specific activity and memory trace must confirm it. An execution audit, recall criterion, memory evidence, and whole-campaign source identity are separate gates before comparative claims.

6.4. Principal parameters

All principal workflows used Euclidean distance, perplexity or neighborhood size 30, seeds 4, 17, and 42, and four CPU threads or one GPU. The revised **fastEmbedR** t-SNE protocol uses compact support ($k = P = 30$), 250 exaggerated and 750 normal iterations, learning rate $\max(n/12, 200)$, and exaggeration 12. **Rtsne** uses 1,000 iterations, learning rate 200,

PCA layout initialization, and no PCA feature preprocessing before its VP-tree workflow. FIt-SNE uses 1,000 iterations, automatic learning rate, PCA layout initialization, and Annoy. **fastEmbedR** fuzzy and binary UMAP used 500 epochs below 10,000 rows and 200 otherwise, with a documented 300-epoch high-variability profile; **uwot** used its default or `fast_sgd = TRUE` schedule.

The complete method-by-method initialization and optimization settings are in the supplement and machine-readable campaign manifest. "Policy matched" means the same class of initialization, not identical starting coordinates; complete-call timings include each method's own initialization.

7. Benchmark results

7.1. Workflow runtime and quality

Campaign `20260930T203319Z` fitted the complete matrix for each of the 11 data sets. On MNIST, median **fastEmbedR** R public-call times were 1.401 seconds for t-SNE and 1.030 seconds for fuzzy UMAP. Direct-Python cuML fit-only times were 2.394 and 0.867 seconds, respectively. These measurements have different boundaries and do not define a speed ratio. Likewise, t-SNE support and independently computed initial coordinates differ between implementations. Figures 1, 2, and 3 show the observed R and Python times within each method family. Circles mark CPU and triangles mark CUDA; fastEmbedR retains one color across panels. Shape does not encode the timing boundary, and mixed R-call and Python-fit boundaries do not define cross-scope speed ratios. Missing points are never filled from another campaign. The figures remain diagnostic because the campaign audit failed; frozen-source replacements are still required for final publication claims.

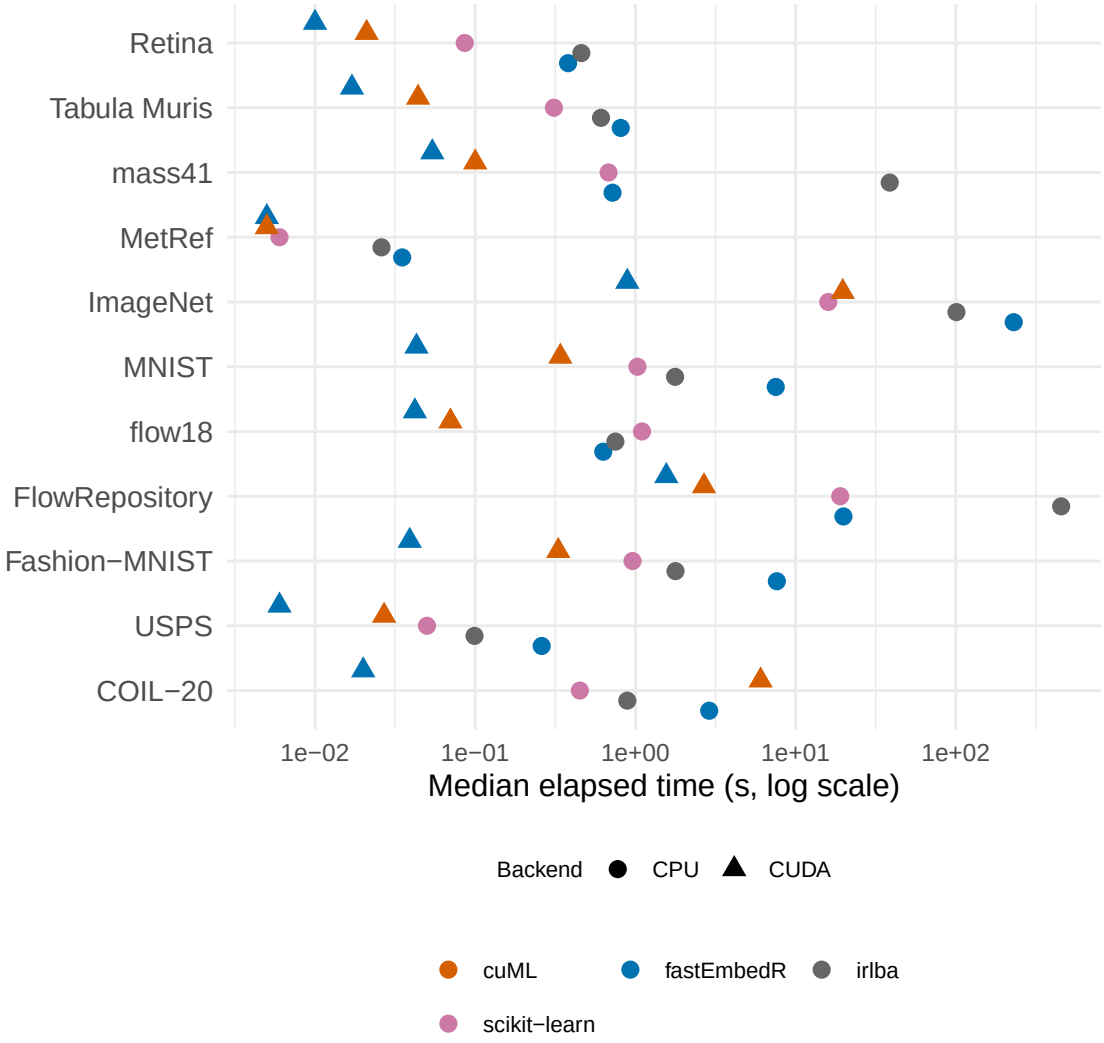


Figure 1: Observed PCA elapsed times in diagnostic campaign 20260930T203319Z. Circles denote CPU and triangles CUDA; fastEmbedR has the same color in all three figures. Points show archived medians without quartile intervals. The supplementary workflow table identifies R-call versus Python-fit boundaries, single fits, and failures; shared axes do not imply a cross-scope speed ratio.

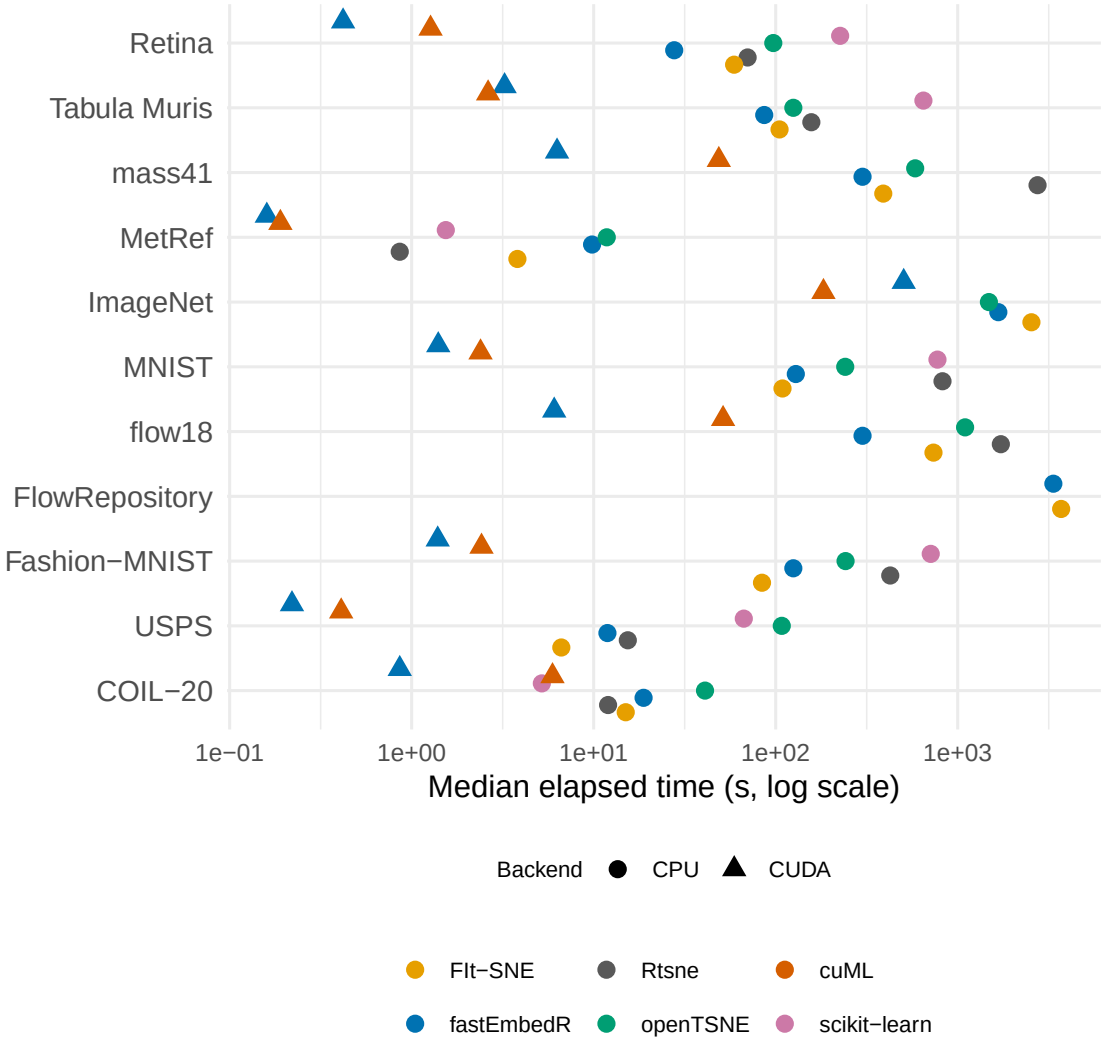


Figure 2: Observed two-dimensional t-SNE elapsed times in the same full-data diagnostic campaign. Circles denote CPU and triangles CUDA; points show archived medians. R public-call and direct-Python fit measurements remain non-comparable timing boundaries. Single fits and missing results are identified in the supplementary workflow table.

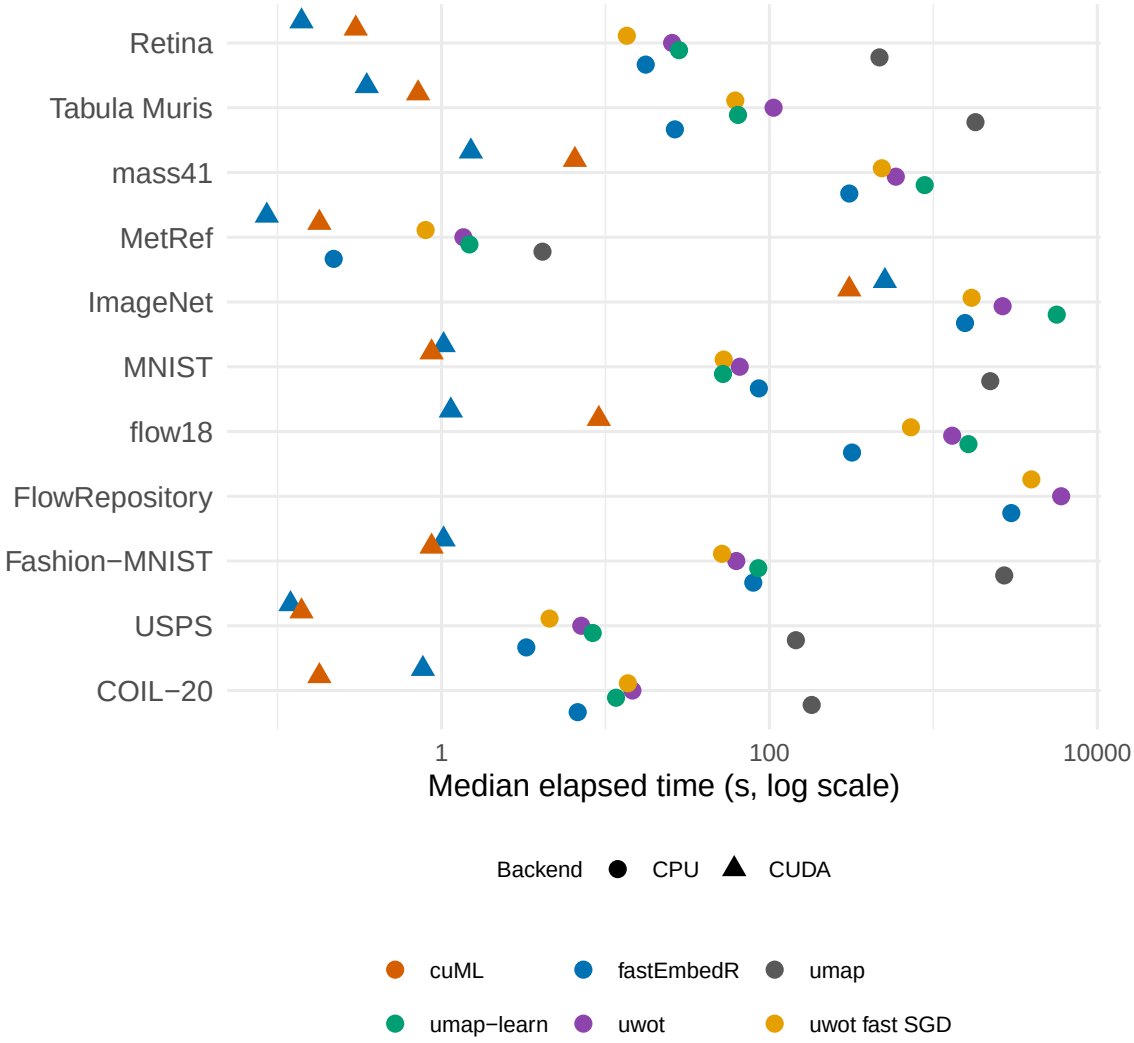


Figure 3: Observed two-dimensional UMAP elapsed times in the same full-data diagnostic campaign. Only fuzzy fastEmbedR UMAP is plotted; binary UMAP remains a separate sensitivity analysis in the supplementary table. Circles denote CPU, triangles CUDA, and points show archived medians. Timing boundaries and missing-result codes follow Figure 1 and the supplementary workflow elapsed-time table.

Peak host resident memory and sampled CUDA device-memory increases are reported in the supplementary peak-memory table. They cover the complete worker, including preparation and quality assessment, rather than the direct-Python fit-only boundary. Short GPU peaks may escape sampling.

The campaign used a 7,200-second per-method limit. Its final audit is `FAIL`, with 34 failed or timed-out tasks across the campaign and four output-audit failures. The supplementary workflow elapsed-time table distinguishes timeout, memory limit, other failure, and completed single fit. Some CUDA memory peaks were not sampled adequately; missing values remain missing rather than zero. Publication-level timing claims remain unresolved.

ImageNet is treated primarily as a runtime and memory stress test. Every workflow fit in this campaign used the complete source matrix; source and fitted row counts are displayed in Table 5.

Observed recall@30 was measured against exact full-reference neighbors. The CUDA routes reached at least 0.9982; the CPU ImageNet and retina HNSW routes reached 0.9672 and 0.9831, below the target 0.99. These sampled checks are diagnostic and cannot be attributed to the optimizer.

Diagnostic median host peak RSS across complete CPU comparator workers was 1.53 GiB for **fastEmbedR** t-SNE and 1.50 GiB for fuzzy UMAP, compared with 1.38 GiB for **Rtsne** and 5.32 GiB for the R **umap** package. Device-wide median incremental memory in sampled CUDA workers was 1.03 and 0.78 GiB for **fastEmbedR** t-SNE and UMAP, versus 1.28 and 0.73 GiB for direct cuML fits. These are whole-worker and sampled-device observations, not fit-only or process-level memory ratios.

Across the ten data sets with successful full-workflow CPU and CUDA fits, median CUDA-minus-CPU trustworthiness differences were +0.0037 for t-SNE and -0.0027 for fuzzy UMAP; Preserve@30 differences were +0.0020 and -0.0058. These are paired quality observations, not proof of identical objectives or trajectories. Dataset-level values and the CUDA/cuML comparison are in the supplementary backend-quality and CUDA-quality tables, respectively. In the later source-locked CUDA candidate run, the median within-data-set fastEmbedR-minus-cuML differences across 11 shared quality samples were +0.01546 trustworthiness and +0.01223 Preserve@30 for t-SNE, and −0.00346 and −0.00055 for fuzzy UMAP. One cuML t-SNE fitted KL was invalid despite finite sampled quality scores; its timing was excluded. These are descriptive quality differences under different objectives and fit boundaries, not speedup or equivalence claims.

7.2. Compact-support sensitivity

When t-SNE has exactly P candidate neighbors, its requested perplexity equals the maximum possible perplexity on that support. Conditional probabilities therefore approach uniform weights; reducing support changes the affinity objective, not only its computational cost. In an exact-neighbor affinity diagnostic at $P = 30$, the median normalized conditional entropy was approximately 1.000 for $k = P$ and 0.761 for $k = 3P$.

At $P = 10$, 20, and 30, fixed-neighbor sensitivity experiments generally favored compact support for runtime and neighborhood preservation, whereas wider support produced lower sampled KL for its different affinity objective. The paired data-set counts and iteration budget are given in the supplementary support-width table; these calls are not complete workflows.

7.3. Multicore scaling

On COIL20, MNIST, flow18, and ImageNet, four CPU threads gave median full-workflow speedups of 2.24 for t-SNE and 2.26 for fuzzy UMAP over one thread. Scaling was not monotonic across 1, 2, 4, 8, and 12 threads; stage times and per-data-set efficiency appear in the supplementary CPU-scaling table.

7.4. Apple Metal

The Apple M3 study covered MetRef, COIL20, USPS, MNIST, Fashion-MNIST, Macosko retina, and Tabula Muris, spanning 873–100,102 observations and 50–16,384 variables. Median four-thread CPU-to-Metal speedup was 2.63 (IQR 1.62–2.88) for compact-support t-SNE, 1.97 (1.08–3.04) for fuzzy UMAP, and 3.35 (1.63–5.85) for binary UMAP. Median Metal-minus-CPU trustworthiness differences were +0.0006, +0.0027, and +0.0010. These results establish performance on Apple M3 only. Other Apple Silicon systems are compatibility targets unless separately benchmarked; Intel Macs are not supported by the native Metal backend.

7.5. Landmarking and held-out transformation

A stratified 80/20 held-out experiment covered all 11 data sets. Median query-to-reference Preserve@30 was 0.371–0.374 for t-SNE and 0.343–0.351 for fuzzy UMAP across CPU and CUDA; the reference did not move. This tests new observations, not reconstruction of omitted fit rows. In the separate 20-percent landmark experiment, median landmark/full runtime ratios were 0.41 and 0.77 for CPU/CUDA t-SNE, and 0.34 and 1.29 for CPU/CUDA UMAP. Median host-RSS ratios were near or above one, so landmarking was not a universal memory saving. Detailed quality and memory results are in the supplementary transformation and landmark tables. These failed-audit observations are diagnostic. A separate source-locked 50,000-row flow18 comparison paired full fitting with 5,000 landmarks. Both cases passed their pair-level audit on identical source and quality rows. On the fixed 256-row within-sample check, Preserve@30 was 0.571 versus 0.552 for UMAP and 0.534 versus 0.541 for t-SNE (full versus landmark). Their observed times were 4.703 versus 2.442 seconds and 10.411 versus 2.982 seconds, respectively. These single-case results are not a multi-seed or full-reference quality comparison; the extended campaign as a whole did not pass audit.

7.6. Supporting PCA and graph utilities

On fixed subsets of 11 data sets, CPU and CUDA rank-2 PCA scores had median aligned agreement of 0.999972 and approximately 1.000000 with dense centered singular value decomposition. CPU ImageNet was an outlier (0.783 score correlation). The latest campaign completed the stored-graph clustering comparison on only two data sets, so it does not support a broad clustering-performance claim. PCA accuracy, clustering denominators, and graph-construction scope are documented in the supplementary PCA-accuracy and clustering tables. Separately, streamed CPU PCA completed a one-billion-row Deep1B score file in 6,924 seconds with 48 GiB of requested job memory and 431 MiB measured peak process RSS. The two-column float32 file has exactly one billion row slots; seven dispersed rows checked after the run were finite. This is a completed PCA stage, not a billion-row UMAP or t-SNE fit, and is not a whole-file finite-coordinate audit.

7.7. Status of the file-backed study

The Deep1B study is a separate experimental campaign, not a replacement for the 11-data-set comparator study. Its one-million-row classic and file-backed arms, larger file-backed arms, and two-device PCA measurement must be reported together only after their individual outputs and the source-locked campaign have passed audit. Execution success alone is not a nearest-neighbor quality pass: sampled recall below the specified gate remains visible even when an embedding and its plot are produced. A two-device PCA calculation that finishes without adequate observations from both GPUs is not evidence of multi-GPU acceleration. Two UCT jobs did complete all requested full-row methods on two L40S devices at one and 10 million rows, with complete output-row scans and positive work recorded on both devices. Both retained recall warnings: four of 128 sampled queries fell below the per-query target. Their execution passes therefore do not establish a qualified speedup or an independent recall holdout.

Consequently, this draft does not report Deep1B runtime ratios, scaling factors, or full-row embedding figures as final results. The completed one-billion-row PCA stage is reported separately because the same campaign had five failed and 21 blocked file-backed cases. The protocol and currently unsupported routes are documented in the supplement so that completed, audited measurements can be inserted without changing the comparison boundary. The newly requested 100-million-row maximum cannot by itself establish an input-larger-than-GPU-memory claim, and remains outside the current source-locked matrix until restart and storage gates pass.

Figure 4 reserves a place in the main paper for the eventual billion-row visualization. Its current panels are *provisional 10-million-row results*, not billion-row fits: they show PCA, t-SNE, and UMAP for the same Deep1B source window, beginning at row 1,000,001, on four CPU cores and two L40S GPUs. Both runs produced all requested output rows, but their nearest-neighbor audits warned that some sampled queries fell below the per-query 0.99 recall target. The figure must be replaced and re-audited before any billion-row claim.

Table 6 reserves the matched-input comparison. Here **XXX** means not yet verified for this manuscript, not zero or an observed value. Each cell must be filled from one frozen source and a passing campaign audit, or replaced by an explicit failure or unsupported code. The supplementary result slots separate stage time, search quality, memory, output coverage, and hardware use.

8. Discussion

fastEmbedR demonstrates that a single R package can expose multicore CPU, Apple Metal, and NVIDIA CUDA implementations of fuzzy UMAP and compact-support interpolation-based t-SNE while maintaining one typed interface. The strongest engineering gains come from reusing sparse structures, using float32 throughout the numerical payload, avoiding repeated R-native copies, retaining accelerator state, pooling temporary memory, and reducing kernel and synchronization overhead. The package does not introduce a new UMAP or t-SNE objective; its contribution is a native, validated, and backend-aware implementation and workflow.

The full-data workflow campaign is insufficient for a release-level speed claim. Its failed final audit, incomplete memory sampling, and mixed R/Python timing boundaries preclude reliable

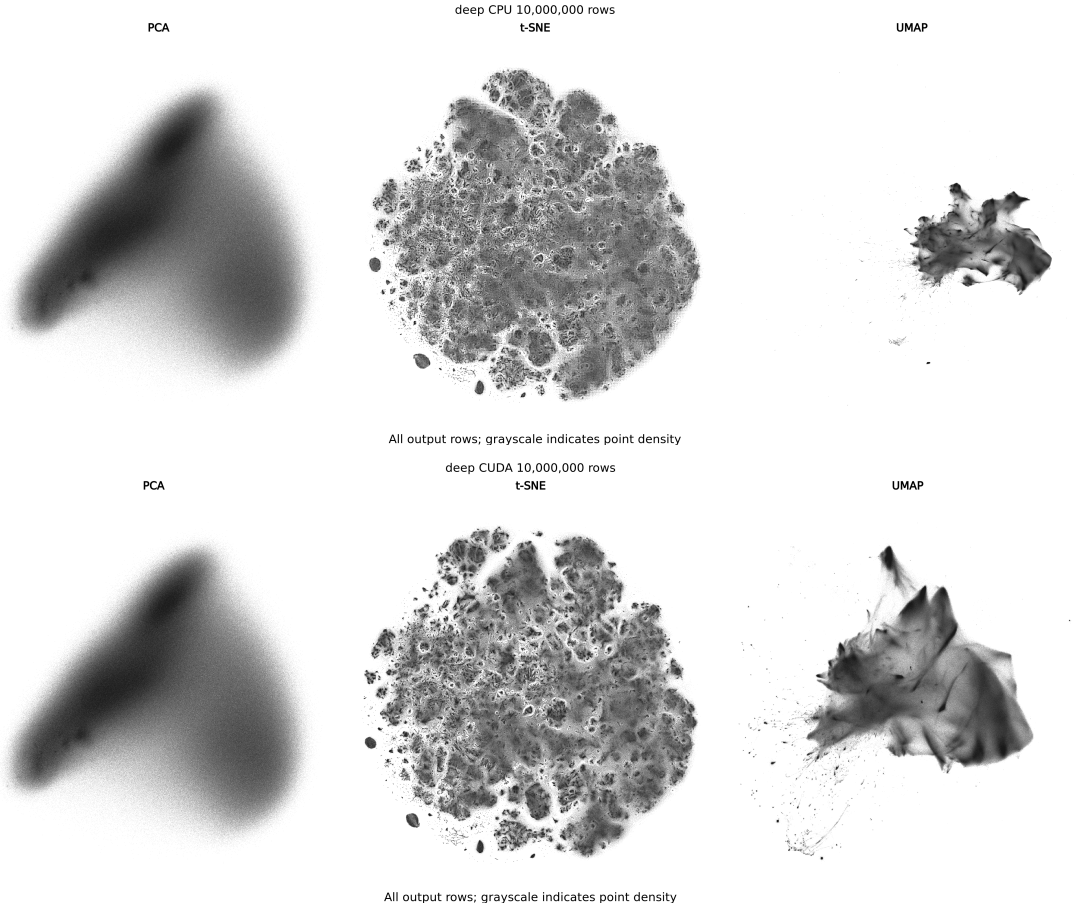


Figure 4: **Provisional Deep1B visualization: 10 million of one billion source rows, not a billion-row fit.** File-backed full-row PCA, t-SNE, and UMAP on four CPU cores (top) and two L40S GPUs (bottom). Grayscale shows output-point density; Deep1B class labels were not available. Both runs completed with a per-query nearest-neighbor recall warning. These panels are layout placeholders for the deferred billion-row experiment and do not establish its quality, runtime, or memory behavior.

Table 6: Reserved matched one-million-row Deep1B results. Time covers the complete fit boundary; scratch excludes the shared input and container. GPU peak is the observed peak on the allocated device. XXX cells are unverified placeholders, not measurements.

Method	Route	Backend	Time (s)	RSS (GiB)	GPU (GiB)	Scratch (GiB)	Recall@30
PCA	Classic	CPU 4	XXX	XXX	–	XXX	–
PCA	File-backed	CPU 4	XXX	XXX	–	XXX	–
PCA	Classic	CUDA 1	XXX	XXX	XXX	XXX	–
PCA	File-backed	CUDA 1	XXX	XXX	XXX	XXX	–
t-SNE	Classic	CPU 4	XXX	XXX	–	XXX	XXX
t-SNE	File-backed	CPU 4	XXX	XXX	–	XXX	XXX
t-SNE	Classic	CUDA 1	XXX	XXX	XXX	XXX	XXX
t-SNE	File-backed	CUDA 1	XXX	XXX	XXX	XXX	XXX
UMAP	Classic	CPU 4	XXX	XXX	–	XXX	XXX
UMAP	File-backed	CPU 4	XXX	XXX	–	XXX	XXX
UMAP	Classic	CUDA 1	XXX	XXX	XXX	XXX	XXX
UMAP	File-backed	CUDA 1	XXX	XXX	XXX	XXX	XXX

cross-method ratios, despite useful stage and quality observations. A workflow ratio combines implementation, search policy, precision, initialization, and hardware; it does not isolate optimizer speed. Complete R calls and direct-Python fit times answer different questions and are not pooled. The subsequent source-locked paired CUDA run produced 66 full-row layouts across 11 data sets and six methods, but its final audit was `WARN`: cuML t-SNE reported zero native fitted KL on FlowRepository, although its independently sampled KL was positive; its timing was excluded. Descriptive paired quality differences are reported in the supplement. The extended campaign has ended without a passing final audit: one CUDA worker failed, another CUDA quality task and a long CPU quality task were cancelled, and its controller did not complete. The Deep1B ledgers likewise contain audited passes alongside failures and missing case audits. Neither campaign supports a release-level cross-backend speed claim. Further UCT GPU submissions are suspended pending explicit user and cluster-administrator approval.

Compact support reduces affinity storage and attractive-force work while making conditional probabilities nearly uniform within each selected row. The t-SNE timing results therefore characterize the stated compact-support method, not an affinity-equivalent replacement for conventional wider-support t-SNE. Practical neighborhood quality and KL are reported with runtime for every valid paired data set.

The production payload uses float32, so trajectories need not reproduce a float64 implementation exactly. Accelerator atomics prevent a fixed seed from guaranteeing bitwise GPU reproducibility. Metal performance evidence is currently limited to one Apple M3 system. Native GPU installation depends on matching SDKs, compilers, drivers, linked libraries, and architecture flags. The UMAP interface exposes scientifically central neighborhood, metric, initialization, graph, backend, and seed controls but keeps several optimizer parameters under a validated release policy. Landmarking can reduce CPU work but is not guaranteed to accelerate a resident GPU pipeline or preserve local quality on every data set. These constraints should guide method selection and reporting.

The file-backed routes are experimental rather than part of the headline workflow comparison. Bounded-memory component tests have processed inputs larger than host RAM, and one streamed PCA stage wrote a billion-row result with a 48 GiB job-memory request. The Deep1B source file was smaller than physical RAM on that node, so this is not a physical-host-RAM boundary test. A converged full-data t-SNE and UMAP comparison on a real beyond-RAM data set is not established here. The current Deep1B design tests identical 1-million-row inputs through classic and file-backed routes and requests a 10-million-row file-backed fit. It does not include a Deep1B full-versus-landmark matched analysis. Multi-GPU row partitioning of PCA or search is distinct from a coordinated multi-GPU embedding; the latter requires its own quality and per-device work audit. Disk capacity and graph-construction time remain important limits even when the input itself can be streamed.

Observed approximate-neighbor recall is another limitation. The lowest CPU recall@30 was 0.9672 on full-source ImageNet, below the target 0.99. The shape-based HNSW policy cannot ensure 0.99 recall across data sets with different neighbor-distance geometry, even at similar matrix shapes. Search recall is consequently reported beside embedding quality; a workflow missing the target is not evidence for a speed advantage at 0.99 recall, and no claim treats ImageNet CPU results as optimizer-only evidence. Strong CPU scaling was also nonmonotonic: eight threads had higher median total-workflow speedup than twelve across the measured data sets. In a separate Deep1B five-million-row pilot, selected HNSW settings passed the

disjoint-query *mean* recall gate (0.99750 on holdout), but 200 of 8,192 holdout queries fell below 0.99 and the minimum was 0.667. This is not a per-query guarantee or evidence that the same settings succeed on the billion-row graph; the Turing holdout remains unavailable.

Backend-native PCA removes a separate preprocessing boundary and can return a t-SNE-ready initialization. Optional graph clustering reuses a graph already constructed for an embedding workflow, but remains logically separate from evidence for UMAP or t-SNE correctness. Walktrap is most appropriate for smaller graphs because its agglomerative stage is CPU-only and has a denser working set than the multilevel Louvain and Leiden paths.

9. Reproducibility, availability, and sustainability

The package source is available under the MIT license. An online tutorial and function reference documents installation, backend selection, and principal workflows. Questions and bug reports are handled through the public issue tracker. Benchmark code, preprocessing instructions, environment recipes, parameter manifests, and aggregation scripts are maintained in a separate benchmark repository. The companion technical manual maps the public functions to their implementation, backend contracts, file formats, recovery behavior, and known experimental limitations. Raw benchmark data, credentials, container images, and restricted inputs are not redistributed.

The corrected campaign launcher records the package-source commit, embedded source-tarball and container hashes, and benchmark-suite manifest. It also records any container-build source patch; a Git commit alone is insufficient to identify a patched installation. Campaign evidence includes session information, compiler and linker commands, hardware identity, seeds, parameters, input checksums, status for every requested row, and repetition-level outputs. Public data are reconstructed from their original sources with versioned scripts. Restricted data must be obtained under the provider's terms; missing data produce an explicit unavailable status rather than substitution.

The diagnostic campaign was `20260930T203319Z`, built from source commit `7f22c52b` with a recorded container-build patch. Its complete source, archive, library, container, and benchmark-suite SHA-256 identifiers are in the supplementary release-identity table; comparator identities and quality-sample row identifiers are in the machine-readable campaign.

Each campaign status inventory records its requested rows. Unsuccessful combinations remain explicit in the machine-readable archives; no speedup ratio is reported as a release-level result from the failed campaign.

The package test suite covers public R behavior and portable native code. Strict hardware jobs verify requested-versus-observed backend identity and run PCA, neighbor search, matrix-input and precomputed-KNN UMAP and t-SNE, held-out landmark projection, graph clustering, and numerical checks on real Metal and CUDA hardware. CPU checks run on macOS, Linux, and Windows. Hardware logs, source packages, binaries, and SHA-256 manifests accompany the benchmark evidence. The replication bundle includes a small deterministic `iris` script that completes on a standard CPU and separate instructions for the large hardware benchmark.

Host-side line coverage is reported separately for R and portable native code. Metal and CUDA device kernels are not credited by host instrumentation; their current evidence consists of strict real-device execution and numerical tests. Device-aware Metal and CUDA kernel coverage is consequently reported as not measured rather than inferred from host-side bridge

execution.

10. Conclusion

fastEmbedR provides native float32 UMAP and compact-support interpolation-based t-SNE for multicore CPU, Apple Metal, and NVIDIA CUDA execution from R. Its matrix, reusable-neighbor, PCA, landmark, transformation, graph-construction, and community-detection interfaces share strict backend reporting and typed metadata. Metal performance evidence is specific to the tested Apple M3 system and is not generalized to all Apple Silicon hardware. Numerical tests support CPU and CUDA PCA projections, graph construction, clustering, affinities, forces, and backend-specific optimization; a separate M3 study supports the Metal route on that system. The diagnostic campaign does not establish release-level runtime reductions. The CPU ImageNet PCA and HNSW-recall outliers show why data-set-level diagnostics must accompany aggregate summaries. The file-backed study remains a separate experimental evaluation until its output, memory, recall, and source audits support full-row performance claims.

Acknowledgments

The authors thank the University of Cape Town ICTS High Performance Computing team for providing a high-performance computing facility for this study (https://ucthpc.uct.ac.za/).

References

Baglama J, Reichel L, Lewis BW (2026). *irlba: Fast Truncated Singular Value Decomposition and Principal Components Analysis for Large Dense and Sparse Matrices*. doi:10.32614/CRAN.package.irlba. R package version 2.3.7, URL https://CRAN.R-project.org/package=irlba.

Blondel VD, Guillaume JL, Lambiotte R, Lefebvre E (2008). "Fast Unfolding of Communities in Large Networks." *Journal of Statistical Mechanics: Theory and Experiment*, **2008**(10), P10008. doi:10.1088/1742-5468/2008/10/P10008.

Chan DM, Rao R, Huang F, Canny JF (2018). "t-SNE-CUDA: GPU-Accelerated t-SNE and Its Applications to Modern Data." *arXiv preprint arXiv:1807.11824*. doi:10.48550/arXiv.1807.11824.

Halko N, Martinsson PG, Tropp JA (2011). "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions." *SIAM Review*, **53**(2), 217–288. doi:10.1137/090771806.

Kobak D, Berens P (2019). "The Art of Using t-SNE for Single-Cell Transcriptomics." *Nature Communications*, **10**, 5416. doi:10.1038/s41467-019-13056-x.

Linderman GC, Rachh M, Hoskins JG, Steinerberger S, Kluger Y (2019). "Fast Interpolation-Based t-SNE for Improved Visualization of Single-Cell RNA-seq Data." *Nature Methods*, **16**, 243–245. doi:10.1038/s41592-018-0308-4.

McInnes L, Healy J, Melville J (2018). "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction." *arXiv preprint arXiv:1802.03426*. doi:10.48550/arXiv.1802.03426.

Nolet CJ, Lafargue V, Raff E, Nanditale T, Oates T, Zedlewski J, Patterson J (2020). "Bringing UMAP Closer to the Speed of Light with GPU Acceleration." *arXiv preprint arXiv:2008.00325*. doi:10.48550/arXiv.2008.00325.

NVIDIA (2026a). "NVIDIA Collective Communications Library: User Guide." Accessed 2026-10-14, URL https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/.

NVIDIA (2026b). "NVIDIA cuVS: A Library for Vector Search on the GPU." Accessed 2026-10-14, URL https://docs.nvidia.com/cuvs/home.

Poličar PG, Stražar M, Zupan B (2021). "Embedding to Reference t-SNE Space Addresses Batch Effects in Single-Cell Classification." *Machine Learning*, **110**, 1211–1231. doi:10.1007/s10994-021-06043-1.

Pons P, Latapy M (2006). "Computing Communities in Large Networks Using Random Walks." *Journal of Graph Algorithms and Applications*, **10**(2), 191–218. doi:10.7155/jgaa.00124.

Rousseeuw PJ (1987). "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis." *Journal of Computational and Applied Mathematics*, **20**, 53–65. doi:10.1016/0377-0427(87)90125-7.

Traag VA, Waltman L, van Eck NJ (2019). "From Louvain to Leiden: Guaranteeing Well-Connected Communities." *Scientific Reports*, **9**, 5233. doi:10.1038/s41598-019-41695-z.

van der Maaten L (2014). "Accelerating t-SNE Using Tree-Based Algorithms." *Journal of Machine Learning Research*, **15**, 3221–3245. URL https://www.jmlr.org/papers/v15/vandermaaten14a.html.

van der Maaten L, Hinton G (2008). "Visualizing Data Using t-SNE." *Journal of Machine Learning Research*, **9**, 2579–2605. URL https://www.jmlr.org/papers/v9/vandermaaten08a.html.

Venna J, Kaski S (2001). "Neighborhood Preservation in Nonlinear Projection Methods: An Experimental Study." In *Artificial Neural Networks – ICANN 2001*, pp. 485–491. Springer. doi:10.1007/3-540-44668-0_68.

Wolf FA, Hamey FK, Plass M, Solana J, Dahlin JS, G"ottgens B, Rajewsky N, Simon L, Theis FJ (2019). "PAGA: Graph Abstraction Reconciles Clustering with Trajectory Inference through a Topology Preserving Map of Single Cells." *Genome Biology*, **20**, 59. doi:10.1186/s13059-019-1663-x.

Affiliation:

[1]Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology (ICGEB), Cape Town 7925, South Africa
[2]Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine, University of Cape Town, Cape Town 7925, South Africa
[3]Computational Biology Group, International Centre for Genetic Engineering and Biotechnology (ICGEB), Trieste, Italy
[4]Life Science Department (DSV), University of Trieste, Trieste, Italy
[5]Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy
[6]Magnetic Resonance Center, University of Florence, Sesto Fiorentino, Italy
[7]Translational Bioinformatics Group, ICGEB, New Delhi, India
[8]Bioinformatics Facility, Department of Cellular, Computational and Integrative Biology - CIBIO, University of Trento, Trento, Italy
*These authors contributed equally.
†Co-corresponding authors:
E-mail: tenori@cerm.unifi.it, stefano.cacciatore@icgeb.org
URL: https://github.com/tkcaccia/fastEmbedR

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd